$\|_\infty$). In subsequent sections, we provide a constructive proof by combining *routing* through attention, *arithmetic* through the FFN, and *composition* via block iteration.

# 4. Model

In this section we define a minimal Transformer variant that admits a direct constructive proof. Our block retains only (i) *selective readout (routing)* via self-attention, (ii) *token-wise local computation (arithmetic/gating)* via an FFN, and (iii) *composition* via depth-wise repetition. In subsequent sections, we isolate these three functionalities as separate lemmas, so that constructing the executor reduces to assembling them.

## 4.1. Architecture

**Assumption 4.1** (Simplified Transformer)**.** We consider a stack of $T$ blocks. Each block consists of single-head self-attention and a token-wise two-layer ReLU FFN, and both modules include residual connections. We do not use LayerNorm or Dropout.

**Why this simplification.** We remove LayerNorm/Dropout to isolate the minimal computational roles used in the construction: selective retrieval (attention), token-wise local computation (FFN), and depth-wise composition. These components already suffice to implement the interpreter blueprint of Section 7; architectural details can be reintroduced once the routing/arithmetic/composition core is fixed. Our goal here is not architectural realism but a clean decomposition that can serve as a proof-level scaffold.

**Dimensions and tokens.** Let the input domain be $\mathcal{X} \subset \mathbb{R}^d$ (compact, as in Section 3), and let $D$ denote the model width. In this constructive section we take the prompt dimension to match the model width, i.e., $d_p = D$.

A prompt is a length-$L$ token sequence (matrix) $P = [p_1; \dots; p_L] \in \mathbb{R}^{L\times D}$ with $p_i \in \mathbb{R}^D$. An input $x \in \mathcal{X}$ is embedded into a single input token $u(x) = E_x x + b_x \in \mathbb{R}^D$, where $E_x \in \mathbb{R}^{D\times d}$ and $b_x \in \mathbb{R}^D$. We also introduce an initial output token $o \in \mathbb{R}^D$ and a *work token* $w \in \mathbb{R}^D$ (initialized to $w = \mathbf{0}$), which serves as a scratch register for intermediate computations. Letting the sequence length be $n = L + 3$, the initial state is

$$Z^{(0)}(P, x) = \begin{bmatrix} p_1 \\ \vdots \\ p_L \\ u(x) \\ w \\ o \end{bmatrix} \in \mathbb{R}^{n\times D}.$$

**Row-wise softmax with temperature.** For $\tau > 0$, let $\mathcal{S}_\tau$ denote the row-wise softmax defined by $\mathcal{S}_\tau(r) = \mathrm{softmax}(r/\tau)$ for each row vector $r$, and applied row-wise to matrices.

**Single-head attention.** Let $W_Q, W_K, W_V \in \mathbb{R}^{D\times D}$. Define single-head self-attention by

$$\begin{aligned}\mathcal{A}_\tau(Z) &:= \mathcal{S}_1\left(\frac{(ZW_Q)(ZW_K)^\top}{\tau\sqrt{D}}\right)(ZW_V) \\ &= \mathcal{S}_\tau\left(\frac{(ZW_Q)(ZW_K)^\top}{\sqrt{D}}\right)(ZW_V).\end{aligned}$$

Unless stated otherwise, we use full self-attention; in a causal variant, the output token can still attend to the prompt prefix.

**Token-wise FFN.** For a hidden width $h$, define the token-wise two-layer ReLU FFN

$$\phi(z) = W_2\sigma(W_1 z + b_1) + b_2, \quad W_1 \in \mathbb{R}^{h\times D},\ W_2 \in \mathbb{R}^{D\times h},$$

where $\sigma$ is applied elementwise. Extend $\phi$ row-wise to matrices by $(\Phi(Z))_{i,:} = \phi(Z_{i,:})$.

**Block update and scalar readout.** The block update is written as

$$\begin{aligned}Z^{(t+\frac{1}{2})} &= Z^{(t)} + \mathcal{A}_\tau\big(Z^{(t)}\big), \\ Z^{(t+1)} &= Z^{(t+\frac{1}{2})} + \Phi\big(Z^{(t+\frac{1}{2})}\big), \qquad t = 0, \dots, T-1.\end{aligned}$$

From the final output token $z_{\mathrm{out}}^{(T)} := Z_{n,:}^{(T)}$, we define the scalar output as

$$F_\theta(p, x) := v_{\mathrm{out}}^\top z_{\mathrm{out}}^{(T)} + b_{\mathrm{out}}.$$

The full parameter set $\theta$ includes all weights above as well as $(E_x, b_x, o, v_{\mathrm{out}}, b_{\mathrm{out}})$.

## 4.2. Viewing the prompt as key–value memory

For the prompt matrix $P = [p_1; \dots; p_L] \in \mathbb{R}^{L\times D}$, define slot-wise keys and values by

$$K_P := PW_K, \qquad V_P := PW_V,$$

where $W_K, W_V$ are the same matrices used inside $\mathcal{A}_\tau$.

Given a query $q \in \mathbb{R}^D$, define the prompt read operation as

$$\mathcal{R}_\tau(q; P) := \mathcal{S}_\tau\left(\frac{qK_P^\top}{\sqrt{D}}\right) V_P \in \mathbb{R}^D.$$

Since $\mathcal{S}_\tau(\cdot)$ outputs a probability vector (nonnegative entries summing to 1), $\mathcal{R}_\tau(q; P)$ is a convex combination of the

rows of $V_P$. Hence, if a sufficient score margin is ensured, taking $\tau$ small yields *near one-hot routing* that concentrates on a single slot (Routing Lemma).

In our constructions, we partition the slots into parameter storage and work memory:

$$[L] = \mathcal{I}_{\text{par}} \dot{\cup} \mathcal{I}_{\text{wrk}}.$$

In summary, the prompt stores a program (parameters/control signals), while attention retrieves the required pieces and feeds them into state updates.

## 5. Main Theorem

This section states the Prompt-UAT objective as a single theorem and summarizes the proof strategy within one page. The key point is that *the weights $\theta^*$ are fixed as an executor*, while *task-specific degrees of freedom are supplied by the prompt $p$.*

### 5.1. Theorem statement

We reuse the input domain $\mathcal{X} \subset \mathbb{R}^d$ (compact) and the prompt space $\mathcal{P} = \mathbb{R}^{L \times d_p}$ from Section 3. Assumption 3.3 ensures $\mathcal{H}_\theta \subset C(\mathcal{X})$ is well-defined.

**Theorem 5.1** (Prompt-UAT (fixed backbone + prompt switching))**.** *Let $\mathcal{X} \subset \mathbb{R}^d$ be compact and let $\mathcal{P} = \mathbb{R}^{L \times d_p}$. Within the simplified Transformer family of Section 4, there exist architecture sizes $(T, D, \text{FFN width})$ and a temperature $\tau > 0$, and there exist fixed parameters $\theta^*$ (including $\tau$ if treated as a parameter) such that the following holds: for every $g \in C(\mathcal{X})$ and every $\varepsilon > 0$, there exists a prompt $p_g \in \mathcal{P}$ satisfying*

$$\sup_{x \in \mathcal{X}} \left| F_{\theta^*}(p_g, x) - g(x) \right| < \varepsilon.$$

*Equivalently, $\overline{\mathcal{H}_{\theta^*}}^{\|\cdot\|_\infty} = C(\mathcal{X})$.*

*Remark* 5.2 (Expressivity versus learnability). Theorem 5.1 is an expressivity (existence) result: it asserts that for each target $g$ there exists a prompt $p_g$ under a fixed executor $\theta^*$. It does not provide an algorithm to find $p_g$, nor does it claim that pretrained LLMs satisfy the theorem as stated. We view optimization and pretraining effects as separate axes that determine which parts of $\mathcal{H}_{\theta^*}$ are practically reachable.

### 5.2. Proof overview

The proof combines *standard UAT* with an *interpreter construction*.

**Proposition 5.3** (Proof blueprint)**.** *Fix $\theta^*$ as in Theorem 5.1. Then for any $g \in C(\mathcal{X})$ and any $\varepsilon > 0$:*

1. ***(MLP approximation)*** *There exists a ReLU MLP $N_g$ such that $\|N_g - g\|_\infty < \varepsilon/2$.*
2. ***(Executor approximation)*** *There exists a prompt $p_g \in \mathcal{P}$ encoding the parameters of $N_g$ such that $\|F_{\theta^*}(p_g, \cdot) - N_g\|_\infty < \varepsilon/2$.*
3. ***(Combination)*** *By the triangle inequality, $\|F_{\theta^*}(p_g, \cdot) - g\|_\infty < \varepsilon$.*

*Proof sketch.* In the first step, we apply the standard universal approximation theorem for ReLU MLPs: for any $g \in C(\mathcal{X})$, choose an MLP $N_g$ satisfying $\|N_g - g\|_\infty < \varepsilon/2$. Next, we construct a fixed executor $\theta^*$. The executor interprets prompt slots as key–value memory, selectively retrieves the required parameter fragments via attention-based routing, uses the token-wise FFN as a local arithmetic module to approximate each layer's linear transformations and ReLU gating, and composes these operations across blocks to unroll the forward pass of $N_g$. By allocating stepwise error budgets for routing and arithmetic, we can control the total accumulated error to be at most $\varepsilon/2$. Finally, applying the triangle inequality yields $\|F_{\theta^*}(p_g, \cdot) - g\|_\infty < \varepsilon$. □

## 6. Key Lemmas: Routing, Arithmetic, and Composition

The executor construction for Theorem 5.1 can be summarized as a combination of three functionalities: (i) attention *almost-selectively reads* a desired prompt slot (routing), (ii) the FFN combines retrieved values with the current state to perform a *local update* (arithmetic), and (iii) when iterated, the *accumulated error* is controlled via an explicit budget (composition).

### 6.1. Routing lemma

We use $\pi_\tau$ for the softmax on a single score vector, and $S_\tau$ for the row-wise extension applied to matrices (Subsection 4.1); they coincide when restricted to one row.

**Definition 6.1** (Softmax with temperature and margin)**.** For $\tau > 0$ and $s \in \mathbb{R}^L$, define

$$\pi_\tau(s)_i := \frac{\exp(s_i/\tau)}{\sum_{j=1}^{L} \exp(s_j/\tau)}, \qquad i \in [L].$$

For $i^\star \in [L]$, we say that $s$ has an $(i^\star, \Delta)$-margin if $s_{i^\star} \geq \max_{j \neq i^\star} s_j + \Delta$.

**Lemma 6.2** (Routing: exponential concentration)**.** *If the $(i^\star, \Delta)$-margin in Definition 6.1 holds, then*

$$1 - \pi_\tau(s)_{i^\star} \leq (L-1)e^{-\Delta/\tau}, \quad \pi_\tau(s)_j \leq e^{-\Delta/\tau} \ (j \neq i^\star).$$

*Hence, if $\Delta/\tau \to \infty$, then $\pi_\tau(s) \to e_{i^\star}$.*

**Definition 6.3** (Prompt retrieval)**.** Let the prompt matrix be $P \in \mathbb{R}^{L \times D}$ with slot-keys and slot-values

$$K_P := PW_K, \qquad V_P := PW_V,$$

and let $q \in \mathbb{R}^D$ be a query. Define the read operation by

$$\mathcal{R}_\tau(q; P) := \pi_\tau(qK_P^\top)\, V_P \in \mathbb{R}^D.$$

**Corollary 6.4** (Copying error)**.** *Let $s(q; P) := qK_P^\top$. If $s(q; P)$ has an $(i^\star, \Delta)$-margin, then for any norm $\|\cdot\|$,*

$$\begin{aligned}
&\big\|\mathcal{R}_\tau(q; P) - V_{P,i^\star}\big\| \\
&\le \big(1 - \pi_\tau(s(q; P))_{i^\star}\big) \cdot \max_{j\in[L]} \big\|V_{P,j} - V_{P,i^\star}\big\| \\
&\le (L-1)e^{-\Delta/\tau} \cdot \max_{j\in[L]} \big\|V_{P,j} - V_{P,i^\star}\big\|.
\end{aligned}$$

*Remark* 6.5 (Where the margin comes from). In the constructive proof, the $(i^\star, \Delta)$-margin is enforced by designing keys so that the intended slot has a strictly larger query–key score than all others (e.g., via nearly-orthogonal keys and aligned queries).

### 6.2. Arithmetic lemma

**Lemma 6.6** (Approximation power of a 2-layer ReLU FFN)**.** *Let $\mathcal{U} \subset \mathbb{R}^{d_u}$ be compact and let $T : \mathcal{U} \to \mathbb{R}^D$ be continuous. For any $\delta > 0$, there exist a width $h$ and a 2-layer ReLU network $\Phi : \mathbb{R}^{d_u} \to \mathbb{R}^D$ such that*

$$\sup_{u\in\mathcal{U}} \|\Phi(u) - T(u)\| \le \delta.$$

*Remark* 6.7. In our constructions, $\mathcal{U}$ is a compact set containing tuples of the form *(retrieved parameter fragments, current state)*, and $T$ is the intended layerwise local update. Lemma 6.6 provides the required token-wise arithmetic module.

### 6.3. Composition lemma

**Lemma 6.8** (Error accumulation under composition)**.** *Let $\mathcal{Z}$ be a normed space and let $G_t, \widetilde{G}_t : \mathcal{Z} \to \mathcal{Z}$. Define the compositions*

$$G_{1:T} := G_T \circ \cdots \circ G_1, \qquad \widetilde{G}_{1:T} := \widetilde{G}_T \circ \cdots \circ \widetilde{G}_1,$$

*and the stepwise errors $\epsilon_t := \sup_{z\in\mathcal{Z}} \|\widetilde{G}_t(z) - G_t(z)\|$. If each $G_t$ is Lipschitz on $\mathcal{Z}$ with constant $L_t$, then*

$$\sup_{z\in\mathcal{Z}} \|\widetilde{G}_{1:T}(z) - G_{1:T}(z)\| \le \sum_{t=1}^{T} \epsilon_t \prod_{s=t+1}^{T} L_s$$

*holds.*

*Remark* 6.9 (Error budgeting). In the executor construction, $\epsilon_t$ decomposes into a routing error (e.g., $O(e^{-\Delta/\tau})$) plus an arithmetic error (the FFN approximation accuracy). By allocating per-step budgets $\{\epsilon_t\}$ so that the bound in Lemma 6.8 is at most a target total error $\eta$, we control the overall executor error.

## 7. Construction: Prompt-Encoded MLP Execution

This section presents the skeleton of the interpreter (executor) construction. The key idea is to first approximate a target function $g$ by a ReLU MLP $N_g$, and then design a fixed Transformer $\theta^*$ that uniformly emulates the forward pass of $N_g$ by reading $N_g$'s parameters stored in the prompt. Throughout this section, the input domain is $\mathcal{X} \subset \mathbb{R}^d$ (compact) as in Section 3. We fix a maximum prompt length $L_{\max}$ (hence the available memory) and represent every prompt as a matrix in $\mathbb{R}^{L_{\max}\times D}$, using zero-padding in unused slots. Let $L \le L_{\max}$ denote the number of nonzero (used) slots.

### 7.1. Encoding: Prompt as Parameter Memory

**Definition 7.1** (Target MLP and boundedness)**.** Let $N : \mathbb{R}^d \to \mathbb{R}$ be a depth-$S$ ReLU MLP, where (by convention) $S$ counts the *input layer*, the $(S-2)$ hidden ReLU layers, and the *output layer*. Let $m_\ell$ denote the width of hidden layer $\ell$ and set $m_0 = d$. Define

$$\begin{aligned}
h^{(0)}(x) &= x, \\
h^{(\ell)}(x) &= \sigma\big(W^{(\ell)} h^{(\ell-1)}(x) + b^{(\ell)}\big), \quad \ell = 1, \ldots, S-2, \\
N(x) &= W^{(S-1)} h^{(S-2)}(x) + b^{(S-1)},
\end{aligned}$$

where $\sigma = \mathrm{ReLU}$, $W^{(\ell)} \in \mathbb{R}^{m_\ell \times m_{\ell-1}}$ and $b^{(\ell)} \in \mathbb{R}^{m_\ell}$ for $\ell \le S-2$, and the output layer satisfies $W^{(S-1)} \in \mathbb{R}^{1\times m_{S-2}}$, $b^{(S-1)} \in \mathbb{R}$. We assume that for some $\Lambda > 0$ all parameters are uniformly bounded in the entrywise max norm: $\|W^{(\ell)}\|_{\max} \le \Lambda$, $\|b^{(\ell)}\|_{\max} \le \Lambda$ for all $\ell \in \{1, \ldots, S-1\}$.

**Running example (one hidden layer).** Consider a depth-$S = 3$ ReLU MLP with one hidden layer:

$$N(x) = a^\top \sigma(Wx + b) + c,$$

which matches Definition 7.1 by identifying $W^{(1)} = W$, $b^{(1)} = b$, $W^{(2)} = a^\top$, and $b^{(2)} = c$. A concrete choice of chunking is to store (i) each row $W_{r:}$ and the corresponding bias entry $b_r$ as one fragment, (ii) the output weight $a_r$ as one fragment per hidden unit, and (iii) the scalar $c$ as a final fragment. Thus $\mathcal{J}$ can be indexed by hidden-unit IDs together with a small tag indicating whether the payload is $(W_{r:}, b_r)$ or $a_r$, plus one slot for $c$; the addressing map $\mathrm{addr} : \mathcal{J} \to [L_{\max}]$ places these fragments into predetermined prompt slots. During execution, the work token retrieves the fragments for unit $r$, computes $\langle W_{r:}, x\rangle + b_r$, applies ReLU, multiplies by $a_r$, and accumulates across $r$; repeating this logic realizes the layerwise macro-step in Definition 7.7.

**Definition 7.2** (Chunking and addressing)**.** Fix a finite index set $\mathcal{J}$ and a chunking map $j \mapsto \mathrm{payload}_j(N) \in \mathbb{R}^q$